

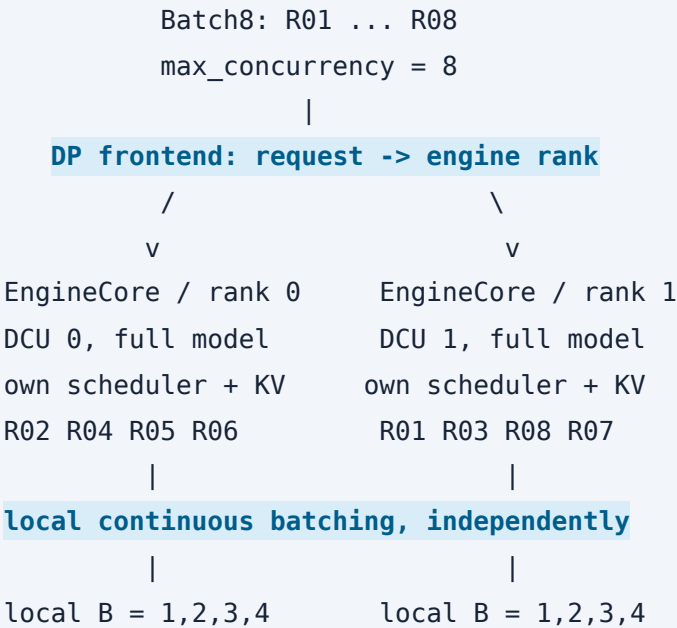
Batch8 如何调度到双卡：一个固定 trace 的完整案例

这份报告用已经验收的 `batch8-dp2-fresh-003` 解释当前优化：8 个并发请求分给两个完整模型副本，每卡 4 个；各卡独立进行 `continuous batching`，长 `prefill` 使用较小的 token 预算，再根据实际单卡批量选择 `kernel`。重点是请求分配与每卡内部调度，kernel 图用于解释调度落到设备上的具体结果。

本例为 Qwen3.5-27B、BF16、gfx936，`DP=2 / TP=1 / PP=1`，每请求输出 1024 token。报告采用 AutoTrace 的 `build-optimization-trace-report` skill，复用已验收 R09/R10 数据。时间来自 R07 原始观测；分析范围覆盖全部 8 个请求的客户端区间，以及每请求首个 `prefill`、首个 `decode` 各 784 个声明 `process` 目标。

1. 8 个请求首先落到两个独立副本

`serve_cscd_dp2.sh` 启动一个对外服务，使用内部 MP 数据并行。每个 EngineCore 有自己的调度器和 KV cache，并在对应 DCU 上运行完整模型。一个请求在选定的副本中推进 `prefill` 和 `decode`。



本例的请求映射固定下来用于解释和可视化。请求表的 `data_parallel_rank_requested` 与实际原生 `kernel` 的 `rank` 全部一致，以下 4+4 分配有完整 `trace` 支持。

设备 / rank	本例请求	输入 token 总数	输出 token 总数	请求数
0	R02, R04, R05, R06	85,843	4,096	4
1	R01, R03, R08, R07	84,781	4,096	4

两卡输入 token 总量分别为 85,843 和 84,781，差值 1,062，相对均值差约 1.24%。全部 8 个请求共同处于客户端在途状态的交集为 **666.185 秒**。这证明本例八并发已形成，4 个请求也都实际落到了各自设备。

正常服务中，若请求没有显式指定 `data_parallel_rank`，`DPLBAsyncMPClient.get_core_engine_for_request()` 计算每个副本的 `score = 4 × waiting + running`，选最小值；并更新本地等待计数，配合协调器约 100 ms 的统计更新。该策略按请求负载选卡；本例使用固定映射来展示它之后的双卡执行过程。

源代码：[服务拓扑](#)、[请求选卡](#)。

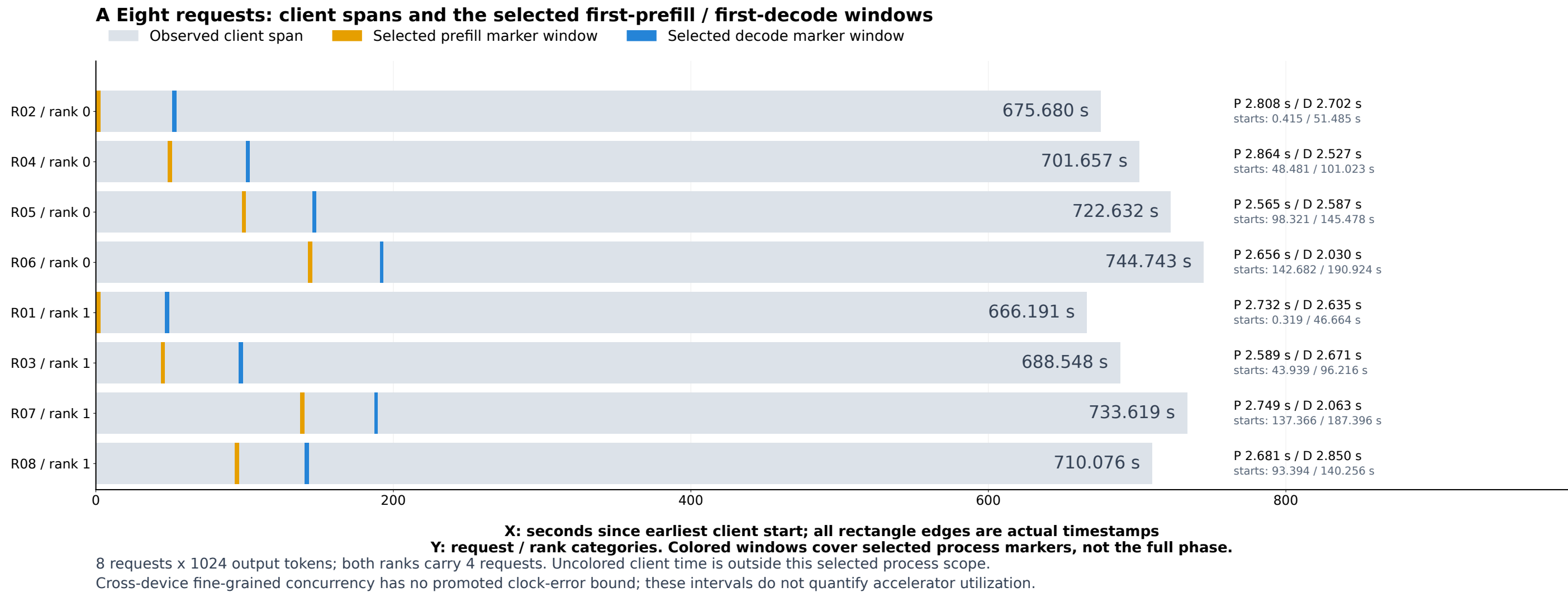


图 A：按 rank 分组的八请求客户端时间线，叠加本例选中的首个 prefill / decode process 窗口。横轴为真实秒数。

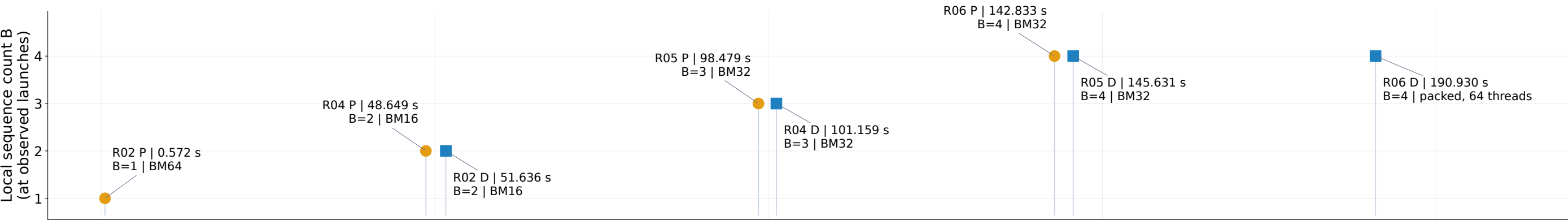
2. 同一张卡上的 batch 怎样从 1 增至 4

把每个声明请求/阶段的首个 GQA 或 packed decode 启动定位出来，可以看到两卡都出现了 **B1 → B2 → B3 → B4** 的实际启动。这里的 B 是这次物理 kernel 接收的序列数：GQA 的 `gridDimZ` 直接给出序列数；packed decode 的 `gridDimY / 48` 给出序列数。

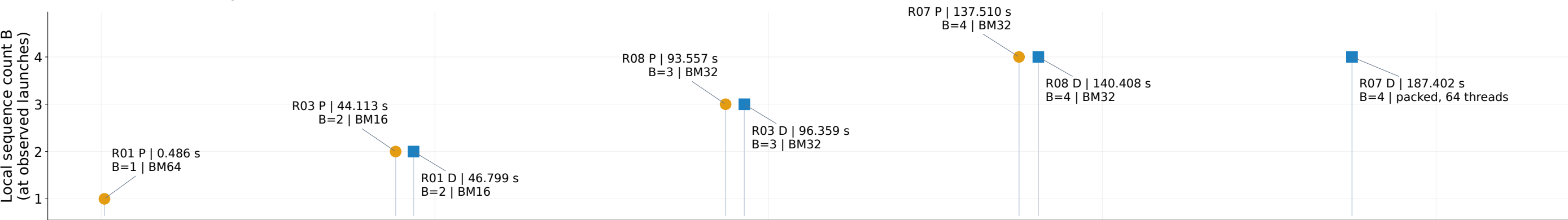
每张卡最早处理一个长请求的 prefill；后续请求开始 prefill 时，已有请求可以同时推进 decode。因此同一个物理 batch 中会出现 prefill 和 decode 混合。R01 的记录名虽然是 decode，但此时 rank 1 的 GQA 启动已经处理两个序列；后面的 R03 decode 记录对应三个序列。这正是“请求自己的阶段”和“整张卡当前 batch”之间的关系。

S How global Batch8 becomes two independent per-device dynamic batches

DCU 0 / DP rank 0 | requests R02, R04, R05, R06



DCU 1 / DP rank 1 | requests R01, R03, R07, R08



Seconds since earliest client start; actual R07 launch timestamps, discrete samples only

P / D are request phase labels. The GQA grid identifies the total sequence count in the physical mixed batch.
Both ranks have observed local B=1,2,3,4 launches. No line interpolates unobserved scheduler states or asserts full-step coverage.
The R06/R07 B4 decode launches use the official packed path; the optimized B1-B3 packed path has no observed hit.

图 S：两卡各 8 个声明阶段的实际启动位置。P/D 是请求阶段；纵轴 B 来自原始 HIP grid。每个点可在下表按 kernel ID 复查。

卡	请求 / 阶段	启动位置 (s)	单卡 B	实际路径 / 配置
0	R02 prefill	0.572	1	GQA BM64
0	R04 prefill	48.649	2	GQA BM16
0	R02 decode	51.636	2	GQA BM16
0	R05 prefill	98.479	3	GQA BM32
0	R04 decode	101.159	3	GQA BM32
0	R06 prefill	142.833	4	GQA BM32
0	R05 decode	145.631	4	GQA BM32
0	R06 decode	190.930	4	packed B4 / 64 threads
1	R01 prefill	0.486	1	GQA BM64
1	R03 prefill	44.113	2	GQA BM16
1	R01 decode	46.799	2	GQA BM16
1	R08 prefill	93.557	3	GQA BM32
1	R03 decode	96.359	3	GQA BM32
1	R07 prefill	137.510	4	GQA BM32
1	R08 decode	140.408	4	GQA BM32
1	R07 decode	187.402	4	packed B4 / 64 threads

上述位置是相对最早客户端开始时间的原始 R07 启动时刻，用 16 个声明阶段的代表启动展示本例；完整 23,660 个 kernel 仍保留在数据表中。图 S 不插值推测两个点之间的调度状态。

3. 调度优化的重点：约束每卡的长 prefill 工作集

源码中的 `Scheduler.schedule()` 从每卡的 running / waiting / skipped-waiting 请求取出尚有 prompt token 未计算的请求，检查其中最大的真实 prompt 长度，再限制该步 token 预算：

当前仍有 prefill 时的最大输入长度	每卡该步 token 预算上限
> 16,384 tokens	512
> 8,192 且 ≤ 16,384 tokens	1024
≤ 8,192 tokens	2048

这是 **token 数的预算**。外部 `max_num_batched_tokens=4096` 和 `max_num_seqs=128` 保留；代码在目标模型/配置命中时限制本步实际预算。纯 decode 没有剩余 prefill 时，不进入这项预算收缩。请求使用已经完成 tokenization 的 `num_prompt_tokens`；调度过程不迁移请求到另一张卡。

本例每个输入为 20,537–22,387 token，全部落入 512 档。首个 prefill 请求标签的 `q_len` 为 192–512；GDN 融合归一化的 672 次原始启动均为 `grid=(1536,1,1)`，对应其输入张量 `T=1536/3=512`。这是物理张量长度，标签中的单个请求 `q_len` 可以更小。

以 MLP 的 gate/up 中间张量为例，源码维度是 `2 × intermediate_size = 2 × 17408 = 34816`。BF16 每元素 2 字节，单个张量的尺寸计算为：

```
4096-token step: 4096 x 34816 x 2 bytes = 272 MiB
512-token step: 512 x 34816 x 2 bytes = 34 MiB
ratio = 1 / 8
```

这说明预算为何能控制大 MLP 中间张量：该张量的理论体积由 272 MiB 变为 34 MiB。它是尺寸计算，整体显存峰值还包含权重、KV、其他中间量和缓存。平台代码还为精确目标配置提供最大 Graph capture size=16 的默认保护；本报告用时间线解释实际调度，不把初始化图内存策略另算成一次已测速度收益。

源代码：[每步调度与三档预算](#)、[Graph 默认配置](#)。

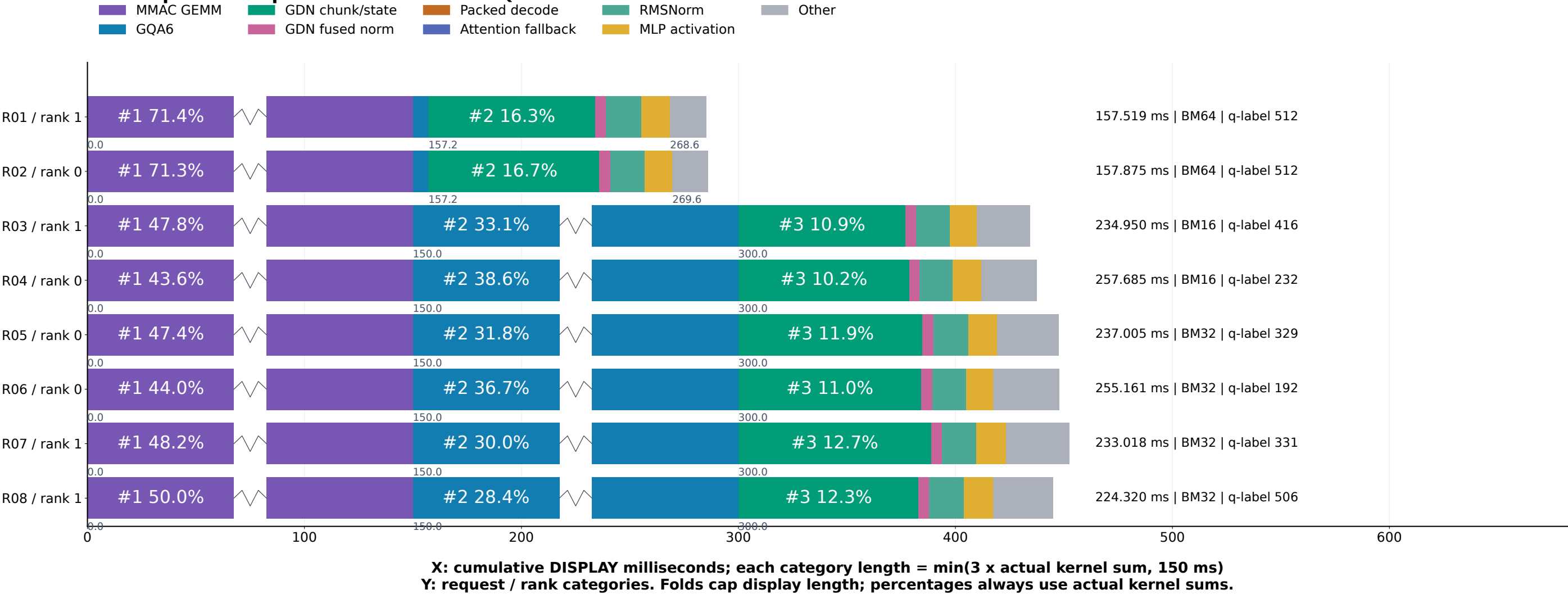
4. 调度怎样改变 GQA 的具体执行

本例 `_gqa6` 共 **224 次**，覆盖全部 8 个请求、14 个请求/阶段单元、两张卡和全部 16 个 full-attention 层。原始严格归属时长合计 **901.614 ms**，占选中范围全部 kernel 时长的 **28.58%**。

其中单卡 B3/B4 命中的 BM32 配置为 **128 次、579.996 ms**，占全体选中 kernel 时长 **18.39%**。这是同一个 GQA kernel 根据当前 batch 选择的配置子集，不能再与 224 次全体相加。

本例物理单卡批量	BLOCK_M：query/head 行数	每 CTA query 位置数	实际线程 / wave64	观察到的启动次数
B1	64	32	256 / 4	32
B2	16	8	128 / 2	64
B3 或 B4	32	16	128 / 2	128

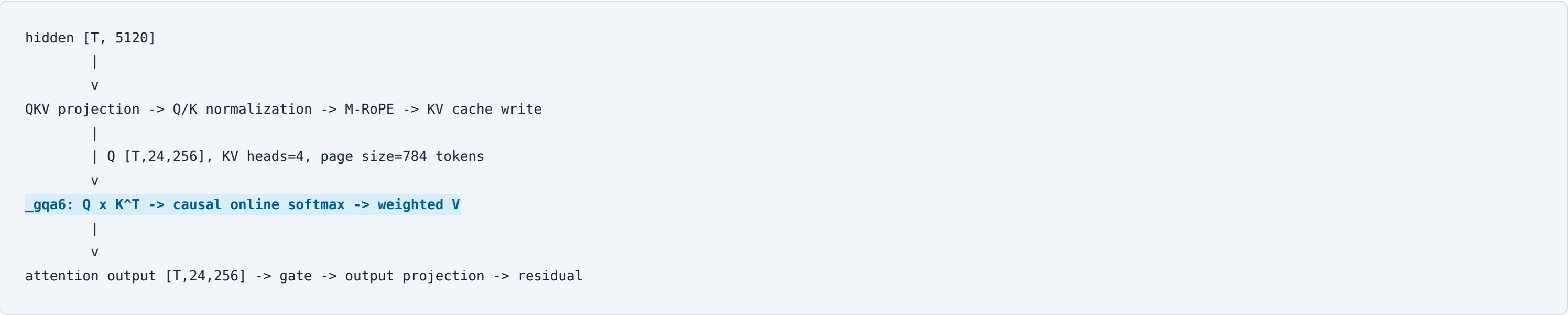
B First-prefill composition: GEMM and GQA6 dominate the selected kernel sum



Each row includes every strict-owned kernel in that declared request/phase unit. Top-5 is ranked independently per row.
Small labels omitted only for fit remain in data/analysis.json and the report tables. Composition order is categorical, not launch order.

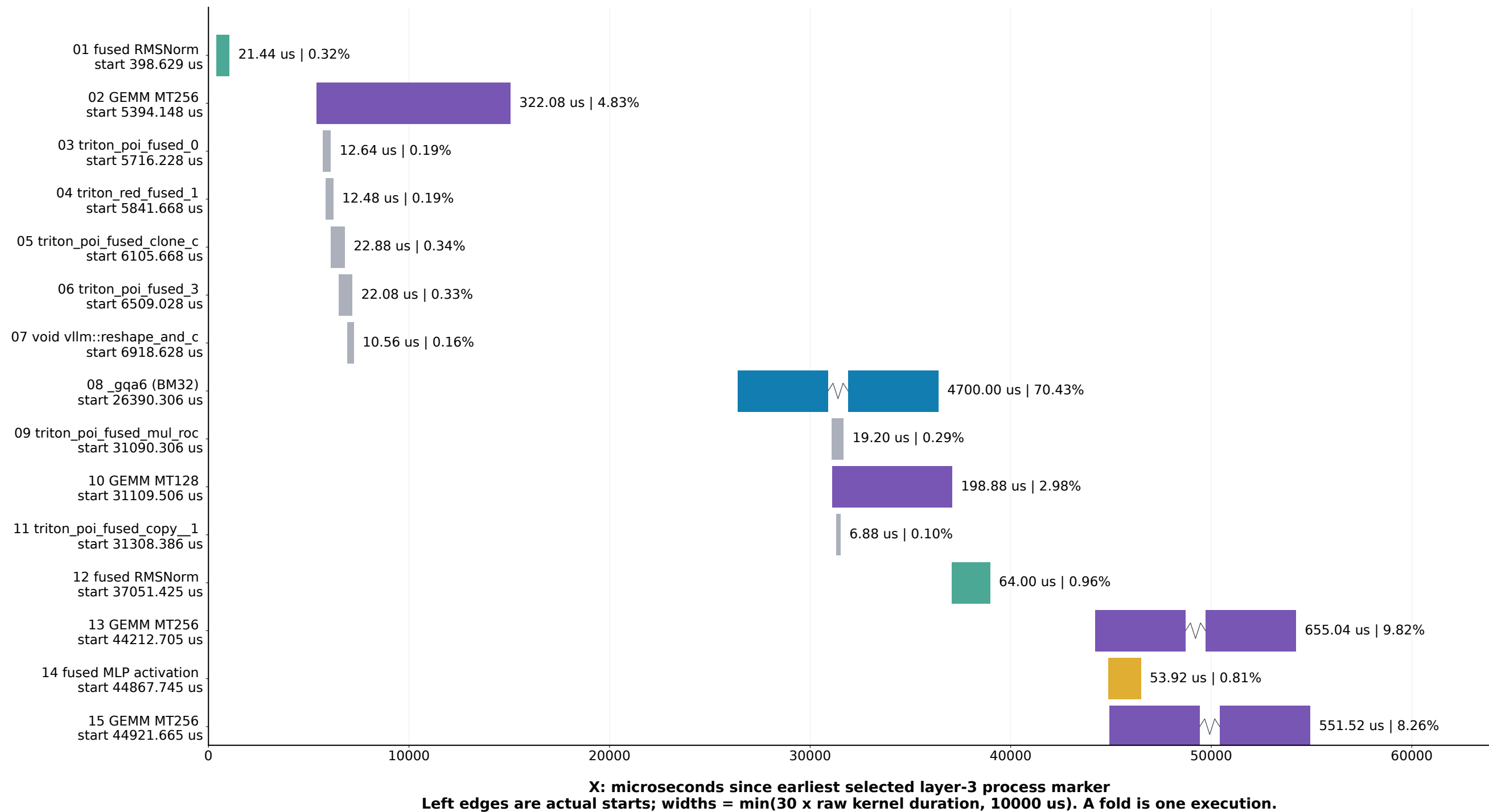
图 B：每请求首个 prefill 的全部严格归属 kernel 组成。各行独立标注最长五类中的可容纳标签，原始总时长和 GQA 配置列在右侧。

在算子链中，GQA 读取当前 batch 的 Q 和分页 KV cache，完成 attention 后输出给门控与输出投影。图 D 选取本例 **R05 / rank 0 / prefill / layer 3**，可直接定位 BM32 的 `_gqa6`：该层所选 kernel 合计 **6,673.60 μ s**，GQA 为 **4,700.00 μ s**，占 **70.43%**。



上图是由固定源码与 FX process 清单连接起来的算子链。kernel 的运行归属仍按原生 launch correlation/index 与精确 process ID 确定。

D R05 / rank 0 / prefill / layer 3: the BM32 GQA6 launch in its actual order



15 strict-owned kernels; additive sum 6673.60 us; enclosing selected marker span 48349.90 us.
Launch order is exact. Display-scaled widths can extend past actual end time. Gaps are not proof of production device idle time.
_gqa6: grid (21,12,3), block 128 threads; 756 launched CTAs, BM32 = 2 query heads x 16 query positions.

图 D: R05 的一个 full-attention 层, 按原始 launch 顺序逐 kernel 展示。左边界保留真实位置, 宽度采用图中声明的放大与折叠。

GQA 的 CTA 数字闭合

R05 的实际启动为 `grid=(21,12,3)`、`block=(128,1,1)`，即 **756 个已启动 CTA**。源码使用 24 个 Q heads、4 个 KV heads：`24/4=6`，每个 KV head 分成 3 个“双 Q-head”组；因此 grid 的 head 维度为 `4×3=12`。

BM32 中，一个 CTA 处理 `2 个 Q heads × 16 个 query 位置 = 32 个 query/head 行`，每行输出 256 个 value 分量。这里的 **32 个 K/V tile token** 是独立的扫描轴。CTA 对每个 query/head 行沿 K/V token 扫描，以 FP32 maximum、denominator 和加权 V 累加器更新 online softmax；最终输出该行 256 个分量。

`grid.x=ceil(max_query_len/16)=21`，覆盖上限 336 个 query 位置；启动值本身将 max_query_len 限定在 321–336。本例请求标签 q_len=329 与此相符。短序列上超出自身 q_len 的 CTA 会提前返回，尾部 query 行受 mask 保护，所以“启动 756 CTA”不表示每个 CTA 都完成相同工作量。`128 threads = 2 waves × 64 lanes`；具体 Triton lane 布局由编译器决定，源码可确定的是上述逻辑 tile 和扫描关系。

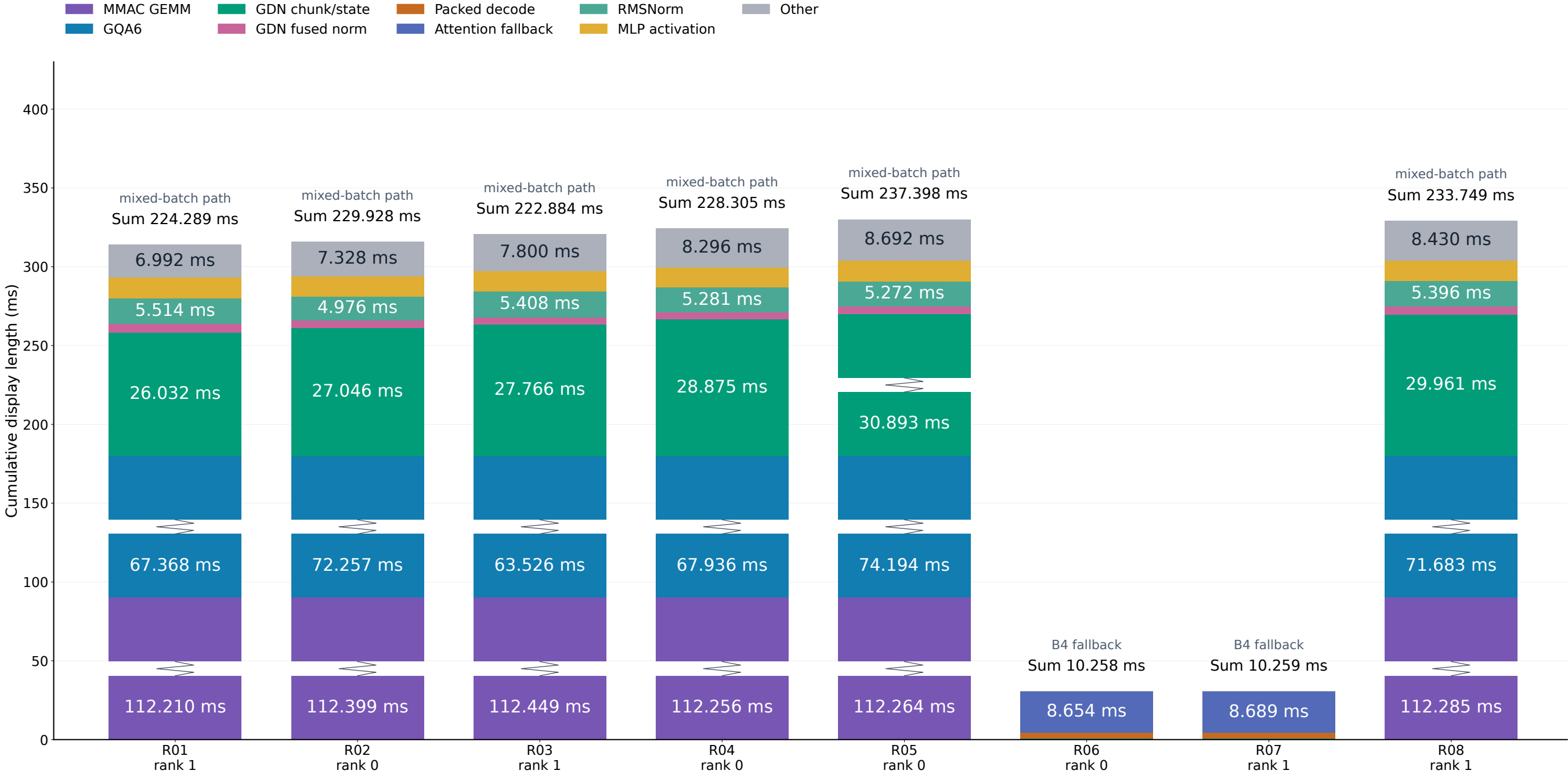
源代码：[GQA grid、page stride 与 online softmax](#)、[运行分发条件](#)。

5. 到 B4 后，decode 使用什么路径

R06 / rank 0 和 R07 / rank 1 的首个 decode 单元使用 `fused_recurrent_gated_delta_rule_packed_decode_kernel`，共 **96 次、2.522 ms**。原始启动参数为 `grid=(4,192,1)`、`block=(64,1,1)`。

`192/48=4` 给出 local B4；64 threads 对应官方一波配置。源码中的 B1–B3 优化使用 4 waves / 256 threads，本例命中数为 0。因此时间线清楚地展示了当前设计：混合 prefill 期间出现 GQA/chunk 路径，到这里的 B4 decode 切回官方 packed 路径。

C First-decode labels: mixed-batch launches and two observed B4 decode fallbacks

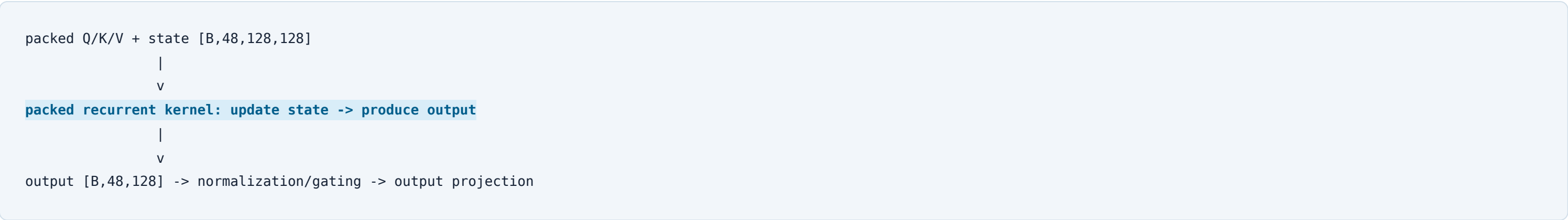


X: request / rank categories; one declared decode unit per request, no averaging
Y: cumulative DISPLAY milliseconds; category height = min(3 x actual kernel sum, 90 ms)

A request labeled decode can share a physical launch with prefill participants: q_len=1 is not the whole kernel workload.
R06/R07 show B4 launches: packed GDN has 64 threads, not the B1-B3 optimized 256-thread configuration.
The difference between about 10 ms and 223 ms is not a measured decode speedup; these units run different physical work.

图 C：每请求首个 decode 标签下的 kernel 组成。R01–R05、R08 包含混合 batch 工作；R06、R07 展示 B4 decode 回退。

R06/R07 单元的选中 kernel 总时长约 10.258 / 10.259 ms，其他单元约 223~237 ms。这个例子中，两种单元的物理 batch 内容不同，正适合解释路径变化；把这些数直接相除不代表 decode 优化加速比。

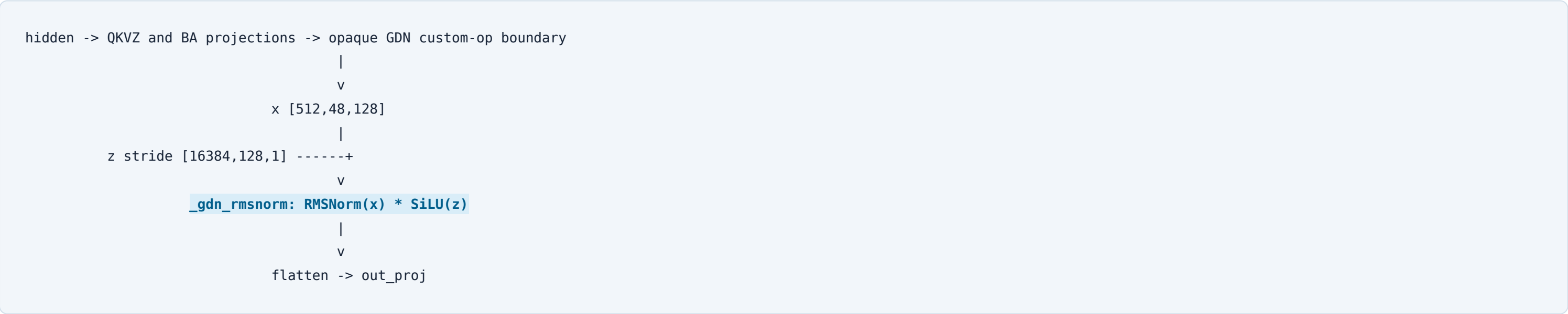


该 B4 启动的 CTA 关系为 4 个 V tiles × (4 sequences × 48 V-heads) = 768 CTA；每 CTA 覆盖 BV=32 个输出 V 分量，并沿 BK=128 的 K 轴规约。总输出 768×32 = 4×48×128 = 24,576 个元素。K/Q heads=16、V heads=48，比例为 3；它与 full attention 的 GQA6 是不同的头轴关系。源码中每个 CTA 更新 [32, 128] state tile，先由 state · K 得到修正量，再更新 state，最后沿 K 轴计算 state · Q 得到 32 个输出；B4 实际 block 为 1×64 lanes。

源代码：[优化 gate 与专用配置](#)、[packed 内核与官方配置](#)。

6. 其余优化在这个例子中的位置

GDN 的 strided RMSNorm+SiLU `_gdn_rmsnorm` 实际命中 **672 次、22.984 ms**，占全部选中 kernel 时长 **0.729%**。源码处理 GDN 输出与 strided gate；实际 owner 是 `gdn_recurrent_core`，按这个 ID 归属，避免再次计入重建名为 `gdn_gated_rmsnorm` 的其他范围。



实际 grid=1536：每 CTA 处理 16 个 head rows、每行 128 元素；1536×16 = 512×48，完整覆盖 token/head 行。256 threads=4×64 lanes，一个 CTA 的逻辑元素数为 16×128=2048。归约沿每行 128 个分量计算 FP32 均方，再乘 weight 和 z×sigmoid(z)，输出形状不变。线程内部的具体分摊与跨 wave 规约布局未从启动参数推断。

GDN chunk h/o 各有 672 次，分别累计 119.514 / 106.372 ms；源码保留了 state/output 复用的实现。本例可证明这些路径实际执行，单独节省的拷贝和时间需要对照数据才能量化。旧单请求自定义 GEMV 在此例中没有匹配事件；M4096 tuning 与 Graph 初始化的收益也不从这些选中阶段外推。

源代码：[Qwen3.5 算子链](#)、[GDN state/output 路径](#)。

7. 本例的耗时组成与后续分析方向

互斥 kernel 类别	命中次数	原始时长和（ms）	占 3,154.602493 ms
MMAC GEMM	4256	1,572.867	49.86%
GQA6	224	901.614	28.58%
GDN chunk/state	6816	388.483	12.31%
GDN fused norm	672	22.984	0.73%
Packed decode	96	2.522	0.08%
Attention fallback	64	17.343	0.55%
RMSNorm	1792	73.919	2.34%
MLP activation	896	60.950	1.93%
Other	8844	113.921	3.61%

对这个例子，双卡分配已经完成 4+4；值得继续分析的是每卡内部的长 prefill 工作集与混合 batch。GEMM 占 49.86%，GQA6 占 28.58%，所以需要把它们放回“本步有几条序列、分配多少 token、是否含 prefill”的调度上下文解释。

R09 的 418 条机会候选中，383 条位于 gdn_recurrent_core；这可作为后续定位入口。过程利用率有 4,631 个可用窗口、7,907 个天然过短窗口、6 个采样缺口；R08 硬件属性中 6,624 项 shape 上下文一致、288 项不一致。使用硬件表时保留这些状态。本报告的主要结论来自固定例子的原始时间与实际 launch，不把候选规则当作已经证明的根因。

8. 在可视化时间线中复查

下载原始 [R10 离线包](#)，解压并打开 acceptance/index.html，进入“完整可交互时间线”。

- 检查分卡：按 rank 0 / 1 查看，分别定位 R02/R04/R05/R06 与 R01/R03/R08/R07。
- 检查 mixed batch：定位 R01 的 decode、layer 3、kv_cache_attention；其 _gqa6 为 local B2。
- 检查 BM32：定位 R05 的 prefill、layer 3、kv_cache_attention；对应图 D 的 4,700.00 μs kernel。
- 检查 B4 decode：定位 R06 或 R07 的 decode、layer 0、gdn_recurrent_core；packed grid 为 (4,192,1)。

本报告 HTML 下方提供请求选择器，可直接查该请求两阶段的精确 kernel ID、rank 和原始纳秒位置。

9. 计算口径与证据索引

每个 kernel 通过 R09 的唯一 `kernel_instance_id` 计一次，使用 `owner_process_range_id` 与 HIP runtime index 复核归属，全部 23,660 个启动参数均可在原始 process context 中找到。时长和 = $\Sigma(\text{end_ns}-\text{begin_ns})$ ；组成占比的分母是该行或该列所包含的实际 kernel 时长和。嵌套 process 时长、双份显示轨道、客户端墙钟都不混入这个分母。

这是一份固定优化版本的例子报告，没有同条件 DP1/旧版对照，因此给出路径、位置和组成。图 A 与图 S 使用真实时间位置；图 B/C/D 的放大、折叠只影响显示，公式写在各图中。R07 的历史原生控制器终止无法追认，其已有离线恢复说明保留；当前 R09/R10 已完成独立验收。

- [报告统计与输入 SHA256](#)、[双卡调度数据](#)、[匹配与计量规则](#)。
- [全量唯一 kernel 表](#)、[原始 HIP 启动参数](#)、[process 表](#)、[请求表](#)。
- [原始 FX process 清单](#)、[源码快照](#)、[图表审计](#)。
- [数据提取代码](#)、[调度分析代码](#)、[图表生成代码](#)、[报告生成代码](#)。

独立验收结果与本报告打包清单在交付时追加到本目录。所有新输出保存在 NFS；原有 R08–R10 封存材料保持原始字节。